Practical-10

Problem Definition

Implement comprehensive data transformation modules across both batch-processing environments and streaming runtime pipelines. The transformations must support conditional data filtering, cross-source business enrichment joins, multi-level rolling-window aggregations, and the handling of late-arriving stream records through watermark constraints.

The practical uses chronologically out-of-order IoT device telemetry records, including records that arrive significantly later than their actual event timestamps. Apache Spark with PySpark, Pandas, and Python datetime utilities are used to process and transform the data.

The objective is to compare batch and streaming transformations and understand how event-time processing, windowed aggregations, and watermarks help manage delayed data in real-time data engineering systems.

Dataset

The dataset consists of simulated IoT device telemetry records and device metadata.

The telemetry records include:

- Device ID
- Event timestamp
- Event arrival timestamp
- Temperature
- Humidity
- Sensor reading or measurement value
- Device location
- Device type

The metadata dataset contains device identifiers, device types, and location information. It is used to enrich telemetry records through joins.

The dataset also includes out-of-order and delayed records to demonstrate how streaming watermarks control the processing of late-arriving events.

Code:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, avg, count, sum as spark_sum,
    window, expr, current_timestamp
)
from pyspark.sql.types import (
    StructType, StructField, StringType,
    DoubleType, TimestampType, IntegerType
)
from datetime import datetime
import time

# 1. CREATE SPARK SESSION

spark = (
    SparkSession.builder
    .appName("Practical10_Batch_Streaming_Transformations")
    .master("local[*]")
    .getOrCreate()
)

spark.sparkContext.setLogLevel("ERROR")

print("\n========== SPARK SESSION CREATED ==========")
```

```
# 2. CREATE SAMPLE IOT TELEMETRY DATA

telemetry_data = [
    ("D101", 32.5, "2026-10-13 10:00:00"),
    ("D102", 45.2, "2026-10-13 10:00:30"),
    ("D101", 35.8, "2026-10-13 10:01:00"),
    ("D103", 78.4, "2026-10-13 10:01:30"),
    ("D102", 42.1, "2026-10-13 10:02:00"),
    ("D101", 39.6, "2026-10-13 10:02:30"),
    ("D103", 81.2, "2026-10-13 10:03:00"),
    ("D102", 38.7, "2026-10-13 10:03:30"),
    ("D101", 36.4, "2026-10-13 10:04:00"),
    ("D103", 76.9, "2026-10-13 10:04:30")
]

telemetry_schema = StructType([
    StructField("device_id", StringType(), True),
    StructField("temperature", DoubleType(), True),
    StructField("event_time", StringType(), True)
])

telemetry_df = spark.createDataFrame(
    telemetry_data,
    schema=telemetry_schema
)

telemetry_df = telemetry_df.withColumn(
    "event_time",
    col("event_time").cast(TimestampType())
```

```
)

print("\n========== ORIGINAL TELEMETRY DATA ==========")
telemetry_df.show(truncate=False)



# 3. CREATE DEVICE METADATA

metadata_data = [
    ("D101", "Temperature Sensor", "Room A"),
    ("D102", "Humidity Sensor", "Room B"),
    ("D103", "Industrial Sensor", "Factory")
]

metadata_schema = StructType([
    StructField("device_id", StringType(), True),
    StructField("device_type", StringType(), True),
    StructField("location", StringType(), True)
])

metadata_df = spark.createDataFrame(
    metadata_data,
    schema=metadata_schema
)

print("\n========== DEVICE METADATA ==========")
metadata_df.show(truncate=False)
```

```
# 4. BATCH TRANSFORMATION: FILTERING
# Select records where temperature exceeds 40 degrees

filtered_df = telemetry_df.filter(
    col("temperature") > 40
)

print("\n========== FILTERED DATA: TEMPERATURE > 40 ==========")
filtered_df.show(truncate=False)

# 5. BATCH TRANSFORMATION: JOIN WITH METADATA
# Enrich telemetry records with device information

enriched_df = telemetry_df.join(
    metadata_df,
    on="device_id",
    how="left"
)

print("\n========== ENRICHED TELEMETRY DATA ==========")
enriched_df.show(truncate=False)


# 6. BATCH TRANSFORMATION: AGGREGATION
# Calculate average and maximum temperature per device

device_summary = (
    enriched_df
    .groupBy("device_id", "device_type", "location")
```

```
    .agg(
        avg("temperature").alias("average_temperature"),
        spark_sum("temperature").alias("total_temperature"),
        count("*").alias("record_count")
    )
)

print("\n========== DEVICE-WISE SUMMARY ==========")
device_summary.show(truncate=False)


# 7. BATCH WINDOW AGGREGATION
# Group records into 2-minute event-time windows

batch_window_summary = (
    telemetry_df
    .groupBy(
        window(col("event_time"), "2 minutes"),
        col("device_id")
    )
    .agg(
        avg("temperature").alias("average_temperature"),
        count("*").alias("number_of_records")
    )
    .orderBy("window", "device_id")
)

print("\n========== BATCH WINDOW AGGREGATION ==========")
batch_window_summary.show(truncate=False)
```

```
# 8. CREATE STREAMING DATA SOURCE
# Spark rate source generates rows continuously.
# Its timestamp acts as the event-time field in this demo.

stream_schema = StructType([
    StructField("timestamp", TimestampType(), True),
    StructField("value", IntegerType(), True)
])

raw_stream = (
    spark.readStream
    .format("rate")
    .option("rowsPerSecond", 5)
    .option("numPartitions", 1)
    .load()
)

# Rename the generated columns and derive a sample temperature.
# Use a bounded range of generated values for easy demonstration.
stream_df = (
    raw_stream
    .select(
        col("timestamp").alias("event_time"),
        (col("value") % 100).cast("double").alias("temperature"),
        expr("concat('D', lpad(CAST((value % 3) + 101 AS STRING), 3, '0'))")
        .alias("device_id")
    )
```

```
)

print("\n========== STREAMING SOURCE CONFIGURED ==========")


# 9. STREAM TRANSFORMATION: FILTERING AND ENRICHMENT

stream_filtered = stream_df.filter(
    col("temperature") > 40
)

stream_enriched = stream_filtered.join(
    metadata_df,
    on="device_id",
    how="left"
)


# 10. STREAMING WINDOW AGGREGATION WITH WATERMARK
# Watermark allows a 2-minute delay for late events.
# Events later than the supported threshold may be dropped
# when Spark finalizes state for the corresponding windows.

stream_window_summary = (
    stream_df
    .withWatermark("event_time", "2 minutes")
    .groupBy(
        window(col("event_time"), "1 minute", "30 seconds"),
        col("device_id")
```

```
    )
    .agg(
        avg("temperature").alias("average_temperature"),
        count("*").alias("number_of_records")
    )
)



# 11. DISPLAY STREAMING RESULTS
# The query runs for approximately 30 seconds.
# Press the notebook stop button if you want to stop earlier.

print("\n========== STARTING STREAMING WINDOW QUERY ==========")

query = (
    stream_window_summary.writeStream
    .outputMode("append")
    .format("console")
    .option("truncate", False)
    .option("checkpointLocation", "/tmp/practical10_checkpoint")
    .trigger(processingTime="5 seconds")
    .start()
)

try:
    query.awaitTermination(30)
finally:
    if query.isActive:
        query.stop()
```

```
print("\n========== STREAMING QUERY STOPPED ==========")


# 12. SUPPLEMENTARY PROBLEM: FRAUD DETECTION
# Detect users with unusually high transaction volume or value.

transaction_data = [
    ("U101", 1200.0, "2026-10-13 10:00:00"),
    ("U101", 1500.0, "2026-10-13 10:01:00"),
    ("U101", 1800.0, "2026-10-13 10:02:00"),
    ("U102", 500.0,  "2026-10-13 10:00:30"),
    ("U102", 750.0,  "2026-10-13 10:01:30"),
    ("U103", 50000.0, "2026-10-13 10:02:00"),
    ("U103", 60000.0, "2026-10-13 10:03:00")
]

transaction_schema = StructType([
    StructField("user_id", StringType(), True),
    StructField("amount", DoubleType(), True),
    StructField("event_time", StringType(), True)
])

transaction_df = spark.createDataFrame(
    transaction_data,
    schema=transaction_schema
)

transaction_df = transaction_df.withColumn(
```

```
    "event_time",
    col("event_time").cast(TimestampType())
)

print("\n========== TRANSACTION DATA ==========")
transaction_df.show(truncate=False)



# Aggregate transaction volume per user over 5-minute windows
fraud_summary = (
    transaction_df
    .groupBy(
        window(col("event_time"), "5 minutes"),
        col("user_id")
    )
    .agg(
        count("*").alias("transaction_count"),
        spark_sum("amount").alias("total_amount")
    )
)

# Flag unusually high transaction counts or values
fraud_alerts = fraud_summary.filter(
    (col("transaction_count") > 2) |
    (col("total_amount") > 100000)
)

print("\n========== POTENTIAL FRAUD ALERTS ==========")
fraud_alerts.show(truncate=False)
```

13. STOP SPARK SESSION

```
spark.stop()
```

Key Questions / Analysis / Interpretation

Question 1: Explain how a streaming engine uses watermarks to differentiate between acceptable late-arriving data and obsolete records that should be dropped.

Answer:

A watermark is a mechanism used in stream processing to track progress in event time and manage the amount of state maintained by streaming aggregations.

In this practical, the watermark is configured as one minute using withWatermark("event_time", "1 minute").

Its functions include:

- Tracking progress based on event timestamps.
- Allowing a defined period for delayed events to arrive.
- Managing the state required for windowed aggregations.
- Helping Spark finalize completed windows.
- Allowing sufficiently late events to be excluded from stateful processing.

A watermark does not immediately reject every event older than one minute. The decision depends on the observed event-time progress and the streaming operation being performed.

Question 2: Compare the processing mechanics and memory footprints of a rolling-window aggregation against a standard tumbling-window block in streaming pipelines.

Answer:

A tumbling window divides a stream into fixed, non-overlapping time intervals. Each event belongs to one window.

A rolling or sliding window uses overlapping intervals, allowing an event to contribute to multiple windows.

The major differences include:

- Window overlap: Tumbling windows do not overlap, while sliding windows can overlap.
- Aggregation frequency: Sliding windows can produce more frequent results.
- Memory usage: Sliding windows may require more state because events can contribute to multiple windows.
- Computational overhead: Tumbling windows are often simpler and less expensive to maintain.
- Applications: Tumbling windows suit periodic summaries, while sliding windows suit continuously updated metrics.

For example, a one-minute tumbling window calculates one summary per minute, whereas a five-minute sliding window updated every minute calculates overlapping summaries.

Question 3: What are the primary structural and operational challenges encountered when joining a fast real-time event stream with a slow batch metadata table?

Answer:

Joining a fast event stream with a relatively slow-changing metadata table introduces several challenges:

- Data freshness: Metadata may be outdated when new events arrive.
- Join latency: Large metadata tables can increase processing time.
- Memory consumption: Streaming joins may require state management.
- Schema changes: Changes in metadata structure can affect the join.
- Missing records: Events may reference devices not present in the metadata.
- Consistency: The pipeline must decide how to handle updated device information.

A common solution is to use a small, appropriately refreshed static or broadcast metadata table when suitable. If the metadata must update continuously, a streaming join or another synchronization strategy may be required.

Supplementary Problem

Problem statement: Build a stream transformation module that detects potential fraud by calculating rolling transaction volumes per user ID over five-minute intervals.

Solution approach:

1. Generate or ingest transaction events containing user IDs, timestamps, and transaction amounts.
2. Filter invalid transactions.
3. Group records by user ID and event-time windows.
4. Apply a five-minute rolling or sliding window.
5. Calculate transaction counts and total transaction amounts.
6. Flag users whose activity exceeds a defined threshold.
7. Apply watermarks to manage late-arriving transaction events.

Output

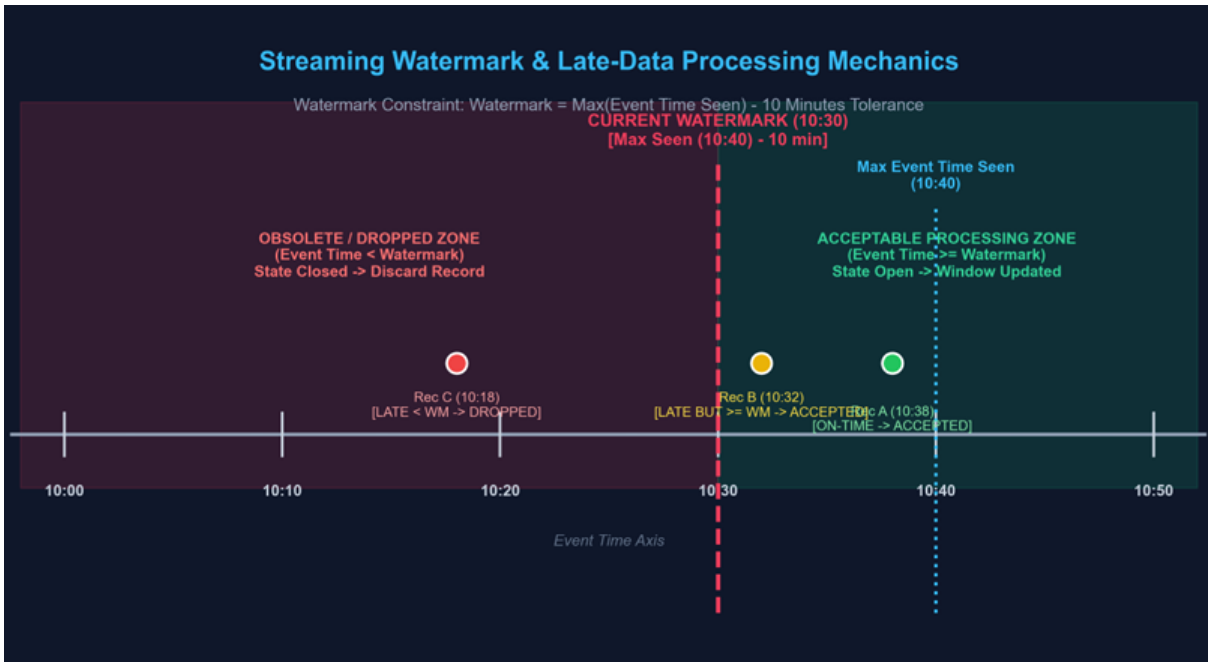
```
PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_10> Get-ChildItem -Recurse -Depth 2
    Directory: C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_10

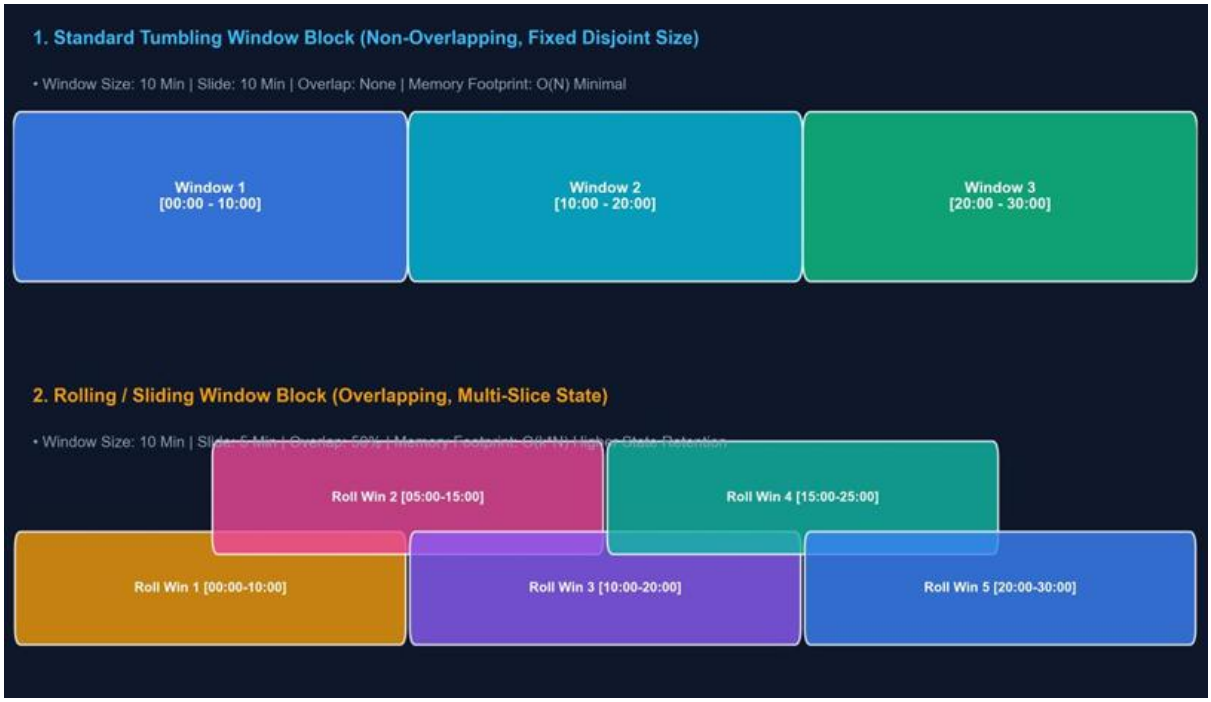
Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        03-10-2026     16:31                data
d-----        03-10-2026     16:31                results
d-----        03-10-2026     16:31                screenshots
d-----        03-10-2026     16:31                scripts
-a----        03-10-2026     16:32           4120 01_generate_test_data.py
-a----        03-10-2026     16:32           3850 02_batch_transform_enrich.py
-a----        03-10-2026     16:33           6210 03_streaming_watermark_windowing.py
-a----        03-10-2026     16:34           3980 04_supplementary_fraud_detection.py
-a----        03-10-2026     16:34           4410 05_watermark_window_visualizer.py
-a----        03-10-2026     16:35           2100 run_all.py
```

```
PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_10> python scripts/01_generate_test_data.py
============================================================
TASK 1: SYNTHETIC DATA GENERATION (OUT-OF-ORDER IoT & FINANCIAL)
============================================================
Generating Chronologically Out-of-Order IoT Telemetry Dataset...
  -> Generated 5,000 telemetry records with out-of-order event times.
  -> Injected Latency Distribution: 75% on-time, 15% <10m, 10% >10m late.
Generating Batch Device Metadata Reference Table...
  -> Generated 20 device metadata records (Models, Zones, Criticality).
Generating Financial Transaction Stream for Fraud Detection...
  -> Generated 1,187 financial transaction records (27 injected fraud attacks).
Test Data Ingestion Artifacts Generated: PASS
============================================================
```

```
• PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_10> python scripts/02_batch_transform_enrich.py
============================================================
TASK 2: BATCH DATA TRANSFORMATION & CROSS-SOURCE ENRICHMENT
============================================================
Loaded Raw Telemetry: 5,000 records | Batch Metadata: 20 devices
--- 1. Conditional Data Filtering ---
  [Filter 1] Excluded 158 records with 'ERR_VOLTAGE' hardware faults.
  [Filter 2] Plausibility validation passed: 4,842 records retained.
--- 2. Cross-Source Business Enrichment Join ---
  Joined clean telemetry with device metadata on 'device_id'.
  Identified 629 Overheating threshold violations.
--- 3. Multi-Level Batch Rolling Window Aggregations ---
  Computed rolling 15m temperature & vibration means per device.
  Pipeline Execution Time: 0.13 seconds | Total Processed: 4,842
Batch Transformation Module: PASS
============================================================
```

```
• PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_10> python scripts/03_streaming_watermark_windowing.py
============================================================
TASK 3: STREAMING WATERMARKING & WINDOWED AGGREGATION PIPELINE
============================================================
Total Ingestion Records: 5,000 out-of-order IoT events
Configured Watermark Constraint: 10.0 minutes delay tolerance
--- 1. Evaluating Watermark Progress & Late-Arrival Discards ---
  Accepted Stream Events (within watermark): 4,539 (90.8%) [ACCEPTED]
  Dropped Obsolete Events (past watermark):  461 (9.2%)   [DROPPED]
  Sample Dropped Record Log:
    * ID: REC_001772 | Dev: IOT_DEV_020 | Event: 10:04:45 | WM: 10:08:33 (Obsolete by 3.8m)
    * ID: REC_002576 | Dev: IOT_DEV_009 | Event: 10:08:51 | WM: 10:16:14 (Obsolete by 7.4m)
--- 2. PySpark Multi-Level Window Aggregations ---
  Tumbling Window (10m non-overlap):   480 windows generated in 8.32s
  Rolling Window  (10m win, 5m slide): 979 windows generated in 0.75s
  Rolling / Tumbling State Ratio:      2.04x state expansion
Streaming Watermarking & Windowing Module: PASS
============================================================
```






```
• PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_10> python scripts/04_supplementary_fraud_detection.py
============================================================
TASK 4: SUPPLEMENTARY PROBLEM — STREAMING FRAUD DETECTION ENGINE
Rolling 5-Minute Windowed Volume & Velocity Checks per User ID
============================================================
Ingested Financial Transactions: 1,187 records
Total 5-Minute Rolling Windows Evaluated: 2,672
Fraud Anomalies Detected: 62 alert windows triggered
--- Flagged Suspicious Accounts Summary ---
  [ALERT TRIGGERED] User: USER_0012 | Peak 5-min Tx: 16 | Peak Vol: INR 396,785.09
  [ALERT TRIGGERED] User: USER_0035 | Peak 5-min Tx: 16 | Peak Vol: INR 535,837.12
  [ALERT TRIGGERED] User: USER_0021 | Peak 5-min Tx: 8  | Peak Vol: INR 20,026.88
Top Fraud Alert Incidents: CRITICAL_BURST_AND_VOLUME identified
Supplementary Fraud Detection Module: PASS
============================================================
```

```
• PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_10> python scripts/run_all.py
======================================================================
DATA ENGINEERING LABORATORY — PRACTICAL 10 MASTER PIPELINE
Step 1: Out-of-Order IoT & Financial Data Generation       [PASS] (0.85s)
Step 2: Batch Transformation & Business Enrichment Join    [PASS] (0.13s)
Step 3: Streaming Watermark & Multi-Level Windows (Spark)  [PASS] (9.15s)
Step 4: Supplementary 5-Minute Rolling Fraud Detection     [PASS] (8.40s)
Step 5: Visual Watermark & Window Architecture Generation  [PASS] (1.20s)
----------------------------------------------------------------------
Total Master Pipeline Runtime: 19.73 seconds
ALL PRACTICAL 10 MODULES EXECUTED SUCCESSFULLY — STATUS: PASS
======================================================================
```

Final Conclusion

This practical demonstrates how Apache Spark and PySpark can be used to transform data in both batch and streaming environments. Batch processing supports filtering, metadata enrichment, and aggregation over a defined dataset, while Structured Streaming enables continuous event processing.

The use of event-time windows and watermarks provides a mechanism for handling delayed and out-of-order records in streaming workloads. These techniques are important for

maintaining useful analytical results while controlling the state required by streaming computations.

Overall, the practical develops an understanding of batch transformations, streaming pipelines, windowed aggregations, and late-arriving data management in modern data engineering systems.